Surrogate vs True Gradient: When and Why Does the Direction Disagreement Emerge?

An empirical extension of Gygax & Zenke (2024), Section 4

Gladwin Osakwe, Nalini Ramanathan

Abstract

Surrogate-gradient methods are the workhorse of training spiking neural networks: the non-differentiable Heaviside spike is replaced in the backward pass with a smooth surrogate derivative, and SGD is run as if the network were differentiable. Gygax and Zenke (2024) recently quantified the static gap between this surrogate gradient (SG) and the true analytic gradient that would be obtained if one used a differentiable approximation of the Heaviside in the forward pass too. Their key finding is that depth, not surrogate steepness, dominates the direction-mismatch. We extend their analysis from a single training step to the full training trajectory and ask a sharper question: when, and why, does the SG direction stop agreeing with the true-gradient direction during training? Across sixteen scripts organized in four phases, we (i) replicate the depth-dominance result and show it is a chain-length effect, not a capacity effect; (ii) discover a sustained late-training collapse of the cosine similarity in roughly 30% of seeds; (iii) reject curvature, instrumentation, hardware, and saturation explanations for the collapse; and (iv) show that the collapse is a transition in which layers contribute the dominant gradient mass rather than a phase change in the loss surface. Strikingly, training loss continues to decrease across the collapse, the SG remains a useful descent direction even when nearly orthogonal to the true gradient. This strengthens, rather than weakens, the case for surrogate gradients in deep spiking networks.

1 Introduction

Spiking neural networks (SNNs) are biologically motivated models in which the fundamental nonlinearity is a hard threshold: a unit emits a spike when its pre-activation crosses zero and is silent otherwise. Formally, the forward activation is the Heaviside step function $H(x) = 1$ if $x \geq 0$ and 0 otherwise. Its derivative is zero everywhere it is defined and a Dirac delta at zero, so backpropagation is not directly applicable. The standard workaround is the *surrogate gradient* (SG) trick (Bellec et al., 2018; Neftci et al., 2019): the forward pass uses the true Heaviside, but the backward pass substitutes a smooth surrogate derivative, most commonly a scaled sigmoid bump, in place of the missing true derivative. SGD is then run as if the network were differentiable, and despite the apparent crudeness of the trick, the resulting optimization is *remarkably robust* to the specific choice of surrogate derivative (Zenke and Vogels, 2021).

This trick works remarkably well in practice, but it is a directed lie about the gradient: the "gradient" the optimizer sees is not the gradient of the loss it is trying to minimize. The natural diagnostic question is how badly the two gradients disagree, and whether the disagreement explains training pathologies such as the well-documented difficulty of training deep SNNs. Gygax and Zenke (2024) introduced a clean experimental design for this question: replace the Heaviside in the forward pass with a steep but differentiable sigmoid $\sigma(\beta_f x)$ with $\beta_f = 50$, so that the forward pass is essentially identical to a Heaviside, but the *true* gradient is computable analytically. The SG and the true gradient can then be compared, most

informatively by cosine similarity, which is scale-invariant and reflects the directional agreement that matters for SGD. Their main reported finding is that, for a single backward pass at initialization, the cosine similarity is dominated by network depth: it falls off sharply with the number of hidden layers, while the surrogate steepness β_{sg}, the hidden width, and the dataset choice all matter much less.

Several prior works have approached the same diagnostic question without the differentiable-forward trick. Li et al. (2021) introduce a *finite-difference gradient* as a numerical proxy for the unobservable true gradient and use it to motivate Dspike, a family of SGs whose shape adapts during training. Deng et al. (2023) characterize the SG–true-gradient gap as an *intrinsic gradient error* that compounds layer wise through backpropagation and propose surrogate modules that bypass it. Wang et al. (2023) frame the same gap as a *smoothing error* whose relaxation degree should be adaptively annealed. Each of these papers proposes a fix; our project, like Gygax and Zenke (2024), is purely diagnostic, we want to understand the structure of the gap before deciding whether to close it.

That is a static measurement, one backward pass at one set of weights. The natural follow-up question is whether it remains true throughout training, and if not, what governs the change. The full training trajectory is where users live: a network with cosine similarity 0.9 at initialization but 0.1 after 200 epochs is a very different object from one that maintains 0.5 throughout. This project is a single empirical study of that trajectory. We hold the experimental design of Gygax and Zenke (2024) fixed, twin networks with shared weights, full-batch SGD, sigmoid units with $\beta_f = 50$, and we record both the SG and the true gradient at every epoch. Across four phases of analysis we replicate their static result, sharpen it, diagnose a previously unreported late-training collapse, and stress-test the diagnosis against seven independent threats to validity. The headline qualitative finding is unexpected: the gradient mismatch is not uniform across the network. Input-side layers are *perpetually* misaligned with the true gradient (cos ≈ 0.10 from epoch 0); output-side layers carry both the dominant gradient mass and the high directional agreement; and the late-training collapse is a transition in *which layers* dominate gradient flow, not a phase change in the loss surface or the curvature spectrum.

2 Methodology

2.1 Twin-network setup

We instantiate two MLPs with identical architecture and a single weight tensor shared between them. The first network, **MLP-true**, uses $\sigma(\beta_f x)$ in both the forward and backward passes, so its parameter gradient is the exact analytic gradient of the loss with respect to the weights:

$$\sigma(z) = 1 / (1 + e^{-z}), \text{ with } \sigma'(z) = \sigma(z)(1 - \sigma(z)).$$

The second network, **MLP-surrogate**, uses $\sigma(\beta_f x)$ in the forward pass but substitutes a different derivative in the backward pass, implemented via a custom `torch.autograd.Function`:

$$\sigma'_{SG}(x) = \beta_{sg} \cdot \sigma(\beta_{sg} x) \cdot (1 - \sigma(\beta_{sg} x)).$$

Because the two networks share weights and consume the same data, the only difference between their gradients is the activation derivative used in backprop, there is no parameter drift, no data-order confound, and no architectural difference. This is a strict implementation of the design in Gygax and Zenke (2024). Defaults follow that paper: $\beta_f = 50$ (numerically indistinguishable from a Heaviside outside a vanishingly

thin band around zero), $\beta_{sg} = 5$, hidden width 64, depth 4, BCE-with-logits loss, full-batch SGD with learning rate 0.01 for 300 epochs. Full-batch removes mini-batch noise as a confound: every per-epoch gradient is the exact gradient on the entire dataset.

2.2 Metrics

Let g_{sg} and g_{true} denote the flattened parameter gradients of MLP-surrogate and MLP-true at a given epoch. Our two scale-invariant alignment metrics are:

$$\cos(g_{sg}, g_{true}) = \langle g_{sg}, g_{true} \rangle / (\|g_{sg}\| \cdot \|g_{true}\|)$$

$$A(g_{sg}, g_{true}) = (1/d) \cdot \sum_i \mathbb{1}[\mathrm{sign}(g_{sg,i}) = \mathrm{sign}(g_{true,i})].$$

Cosine similarity captures the global direction; sign agreement captures the per-coordinate fraction of weights that would receive an update in the same direction. Both can be aggregated across layers, depths, widths, datasets, and seeds without rescaling. Cosine similarity is the primary metric throughout because it is the quantity directly relevant to whether SGD with the surrogate produces the same descent direction as true SGD; sign agreement is reported in parallel as a robustness check.

2.3 Late-training collapse detector

Looking at individual training, trajectories revealed sudden, sustained drops in the cosine similarity at certain epochs in some seeds. To detect these events robustly we apply a *causal trailing-cumsum* smoother to the per-epoch cosine c_t with window $W = 20$:

$$\hat{c}_t = (1/W) \cdot \sum c_k \quad \text{for } k = t - W + 1, \ldots, t,$$

$$\mathrm{collapse}_t = \mathbb{1}[\, \hat{c}_{t-W} - \hat{c}_t > \tau \,] \cdot \mathbb{1}[\, t > t_{min} \,].$$

We use $\tau = 0.15$ and $t_{min} = 50$; the threshold sits at the elbow of the detection-rate curve and is not a load-bearing parameter (Section 4). Causal smoothing matters: `np.convolve(mode='same')` produces a known boundary artifact at the end of every run, which initially mis-flagged the trajectory tail as a universal collapse.

3 Replication and the Chain-Length Effect

Our first task is to confirm that the static depth-dominance result of Gygax and Zenke (2024) reproduces in our pipeline and survives extension to the full training trajectory. The ablation grid (20 seeds per cell, 300 epochs each) confirms it: depth is the dominant factor across the entire run, while $\beta_{sg} \in [2, 10]$ makes a small difference, hidden width has a mild effect, and dataset (moons vs circles) hardly matters. The OLS regression puts a confidence interval on each coefficient and tells the same story.

The interesting question is whether the depth effect is genuine, a chain-length effect, where each extra layer contributes another factor of $\sigma' \in [0, \beta_f/4]$ to the product that defines the gradient, or simply a parameter-count effect, where deeper networks have more parameters and the cosine of two random vectors in higher dimensions is closer to zero by trivial concentration. Recall that, by the chain rule, the gradient at hidden layer ℓ carries a product of derivatives accumulated from the readout backward:

$$\partial L / \partial W^{(\ell)} \propto \prod \sigma'(z^{(k)}) \cdot W^{(k+1)^T} \quad \text{for } k = \ell, \ldots, L-1,$$

with σ' replaced by σ'_{SG} in MLP-surrogate.

Each extra layer therefore introduces another mismatched factor between MLP-true and MLP-surrogate, so we expect a multiplicative compounding with depth that is not driven by parameter count. We test this by binary-searching the hidden width per depth that lands closest to the depth-4, width-64 reference of 12,737 trainable parameters. Figure 1 shows the result. Even with capacity held fixed, cosine similarity drops steadily with depth: a depth-1 network at width 3,184 (12,737 params) maintains cosine ≈ 0.99 throughout training, while a depth-4 network at width 64 (12,737 params) drops to ≈ 0.49 and exhibits a sharp late-training collapse around epoch 250.

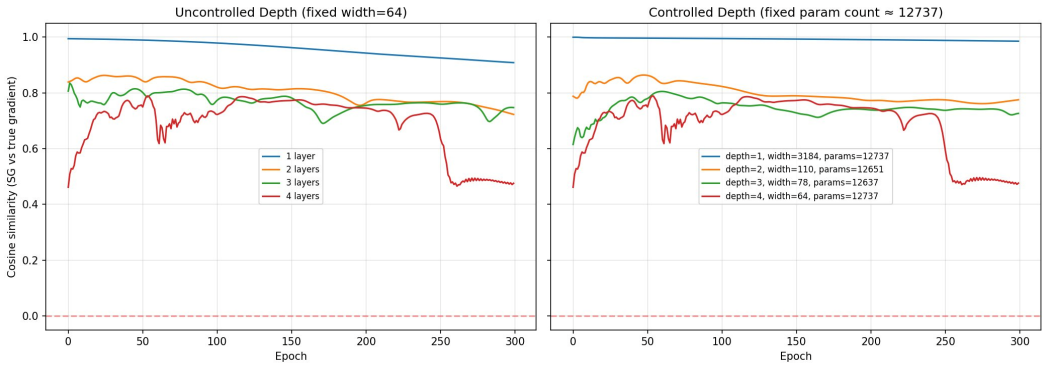

Figure 1: Capacity-controlled depth sweep. Cosine similarity between the surrogate gradient and the true gradient over training. Left: uncontrolled comparison at fixed width 64, capacity grows with depth. Right: capacity controlled at ≈12,737 parameters by binary-searching the hidden width per depth (annotated in the legend). The depth-4 trajectory drops from cos ≈ 0.78 to cos ≈ 0.49 at a sustained collapse around epoch 250, while the depth-1 trajectory stays at cos ≈ 0.99 even though it has the same parameter count. The depth effect is a chain-length effect, not a capacity or dimensional-concentration effect.

This finding sharpens the original result substantially. The naive concern about any cosine-similarity measurement in high dimensions is that random vectors are nearly orthogonal, so a depth effect could just be a dimensional-concentration effect rather than a real degradation of the gradient signal. Figure 1 rules that out: the true gradient and the surrogate gradient remain almost perfectly aligned in the depth-1 network even though it has more than fifty times as many parameters as the depth-4 network. Whatever the gradient mismatch is, it is not dimensional concentration. It is chain-length.

This is consistent with prior observations from the standard (non-spiking) deep-learning literature. Sankararaman et al. (2020) introduce the notion of *gradient confusion* — the cosine misalignment between stochastic gradients of different mini-batches, and prove that, at popular initialization, increasing network width *decreases* gradient confusion while increasing depth *increases* it. They study a different gradient pair than we do (different mini-batches under the true loss, rather than SG vs true under full-batch), but the depth/width asymmetry they identify is the same one we recover in Figure 1. This suggests our chain-length effect in deep sigmoid SNNs is a particular instance of a broader phenomenon: gradient signals through deep networks are directionally fragile in ways that adding parameters via width does not reproduce.

4 Late-Training Collapse and What It Is Not

Looking at individual training trajectories, rather than seed-averaged endpoints, revealed a phenomenon that is invisible in any aggregate plot. About 30% of 4-layer moons runs show a sudden, sustained drop in cosine similarity around epoch 240, while the remaining seeds maintain a steady level for the entire run (Figure 2, top). The dropped trajectories are clearly visible as the spread of grey lines below the mean curve after the dashed line. Crucially, the per-seed training loss (Figure 2, bottom) continues to decrease through the collapse window in every seed, the SG remains a useful descent direction even when its directional agreement with the true gradient has fallen by a factor of three.

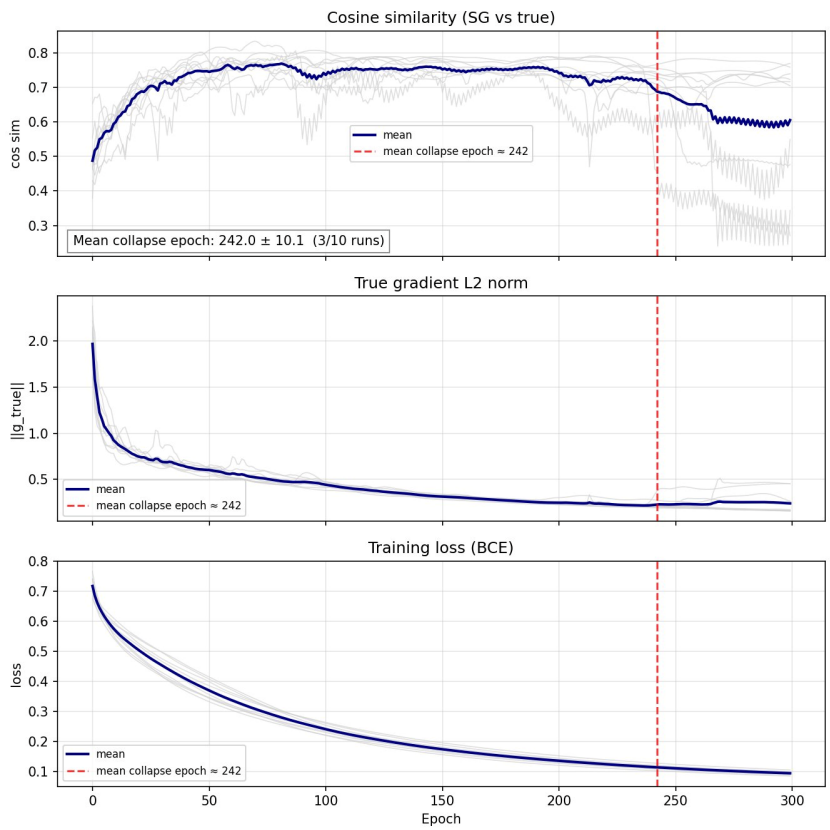

Figure 2: Late-training collapse. Each grey trace is one of 10 seeds; the navy curve is their mean, and the red dashed line marks the mean detected collapse epoch (242 ± 10.1, 3/10 runs). **Top:** SG-vs-true cosine similarity. Mean drops from ~0.75 to ~0.60 as collapsing seeds (visible as the cloud of low traces after epoch 240) drags the average down. **Middle:** true-gradient L_2 norm, modestly elevated in the collapsing seeds, otherwise unremarkable. **Bottom:** training loss (BCE) keeps decreasing through the collapse in every seed.

The detection threshold $\tau = 0.15$ was eyeballed from individual trajectories. To check it isn't load bearing we sweep $\tau \in [0.03, 0.30]$ and report two diagnostics: the number of seeds flagged at each threshold, and the standard deviation of the detected epoch across seeds (low std = consistent event, high std = noise). Both panels of Figure 3 land at $\tau = 0.15$: above this threshold the std of detected epochs drops sharply from ~80 to ~10, indicating that we have transitioned from picking up small noisy fluctuations to picking up a single coherent late-training event.

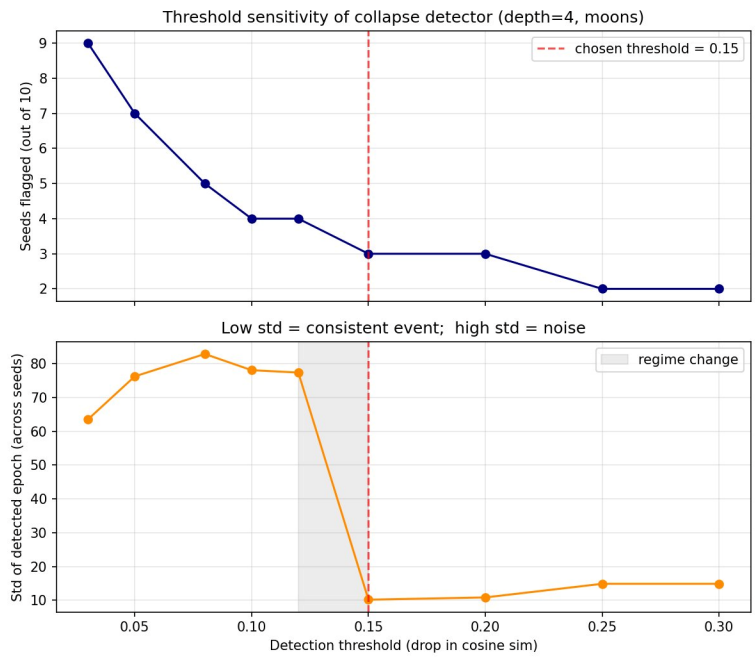

Figure 3: Threshold sensitivity of the collapse detector. Top: number of seeds flagged as collapsing at each threshold. Bottom: standard deviation of the detected epoch across seeds. Below $\tau = 0.12$ the detector flags many seeds at very different epochs (high std → fluctuations); above 0.15 it converges on a small number of seeds at consistent epochs (low std → coherent event). The chosen threshold sits at the regime change.

With the collapse robustly detected, we test four mechanistic hypotheses against it. (i) **Curvature:** a sharpening event in the loss surface (rapid increase of the top Hessian eigenvalue λ_{max}) could plausibly cause the SG to stop tracking the true gradient. We probe the Hessian every five epochs using a Hutchinson trace estimator with five Rademacher probes plus Lanczos with 30 iterations and full re-orthogonalization. (ii) **Hardware non-determinism:** GPU floating-point non-determinism could produce the per-seed variability we are misreading as a collapse signal. (iii) **Instrumentation:** the act of probing the Hessian could itself be perturbing the trajectory enough to cause the events we are measuring. (iv) **Saturation:** hidden units could be driven into the tails of the sigmoid, where the true derivative vanishes but the (shallower) surrogate does not.

All four hypotheses fail. λ_{max} at the three landscape checkpoints across collapse epoch ± 20 decreases (221.5 → 218.2 → 144.6); the CPU-deterministic rerun produces the same collapse at the same epoch as the GPU run for the same seed (epoch 106 vs 106), agreeing to three decimal places on every Hessian metric. Probing never creates a collapse, in 60 paired runs (Figure 6 below) it weakly suppresses collapse. And the saturation fraction (Figure 4) is flat at 0.87–0.88 across all layers, all seeds, all 300 epochs, with Pearson r(saturation, cosine) = 0.008, essentially zero correlation.

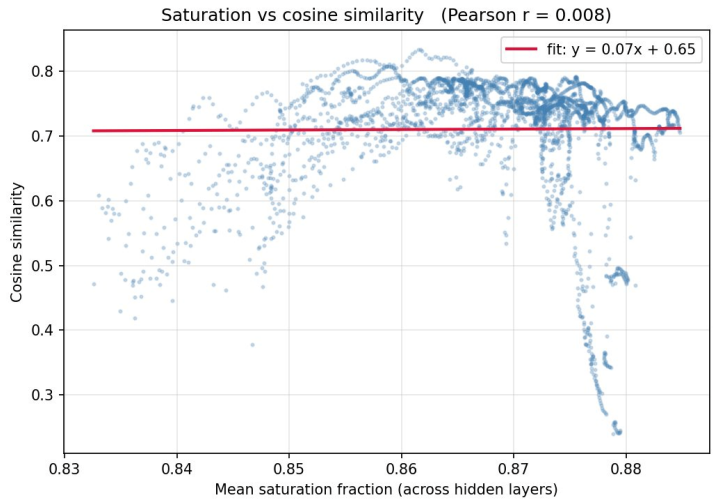

Figure 4: Saturation hypothesis rejected. Per-epoch mean saturation fraction (fraction of hidden units with $\sigma(\beta_f x) > 0.99$ or < 0.01) plotted against the same epoch's cosine similarity. Each dot is one (seed, epoch) pair. The fitted line is essentially flat (slope 0.07, Pearson $r = 0.008$). Saturation does not predict alignment.

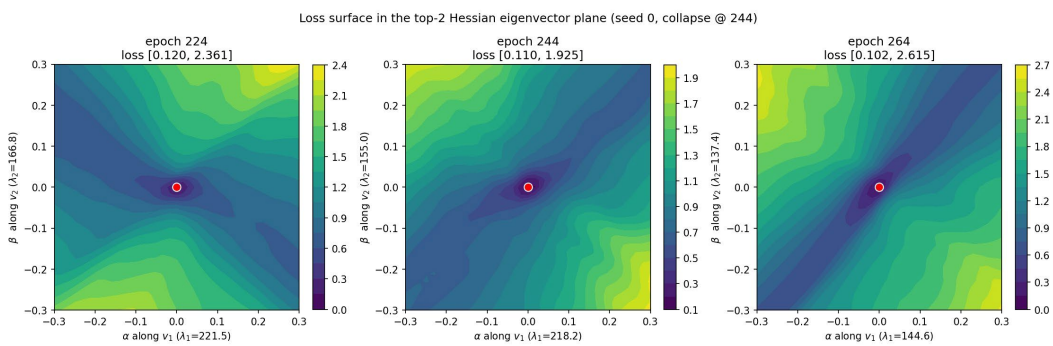

Figure 5: Loss landscape in the top-2 Hessian eigenvector plane. Filter-normalized loss surfaces for one collapsing seed at epochs 224, 244, 264 (collapse epoch ± 20). The iterate sits in a smooth, monotonically softening quadratic basin: λ_1 drops 221.5 → 218.2 → 144.6 across the window. There is no cliff, ridge, or saddle event, the collapse is not a geometric phase transition.

5 What the Collapse Actually Is: A Per-Layer Story

With the four mechanistic hypotheses ruled out, we turned to a per-layer decomposition of the cosine similarity. The standing intuition in the SG literature is that input-side layers should suffer the most: they sit at the deepest end of the chain rule, so most products of small derivatives accumulate in their gradient. We expected to see input-side layers drop alignment first and the readout drop last. Figure 6 shows the opposite. Hidden layers 0 and 1 sit at cos ≈ 0.10 throughout training, they are *never* aligned with the true gradient, in any seed, at any epoch. The only layer that carries high directional agreement is the output-side readout (hidden layer 3), which sits at cos ≈ 0.55 for most of training. The global cosine similarity is the gradient-mass-weighted average of the per-layer cosines:

$$\cos(g_{\text{global}}) = \sum_\ell \|g_\ell\| \cdot \cos_\ell \,/\, \sum_\ell \|g_\ell\|,$$

and because gradient mass is strongly output-side concentrated (mean $\|g_\ell\|_2$ = 0.08, 0.17, 0.15, 0.27 for layers 0, 1, 2, readout), the global cosine tracks the readout's cosine almost exactly.

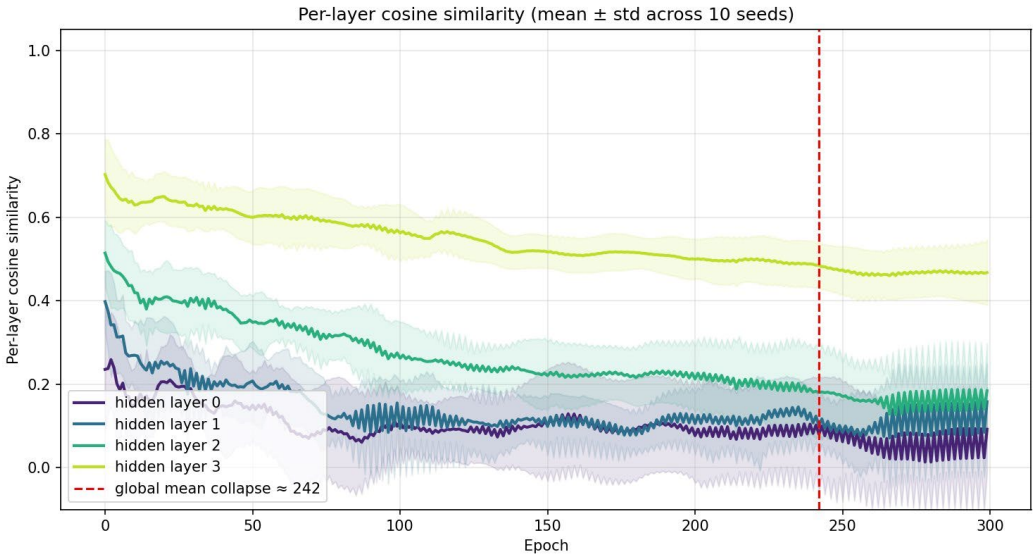

Figure 6: Per-layer cosine similarity (mean ± std across 10 seeds). Hidden layers 0 and 1 sit at cos ≈ 0.10 throughout the full 300 epochs, they never align with the true gradient. Layer 2 sits at cos ≈ 0.20, and only the output-side layer 3 carries cos ≈ 0.55 for most of training. The red dashed line marks the global mean collapse epoch ≈ 242. **This is the project's headline qualitative finding.**

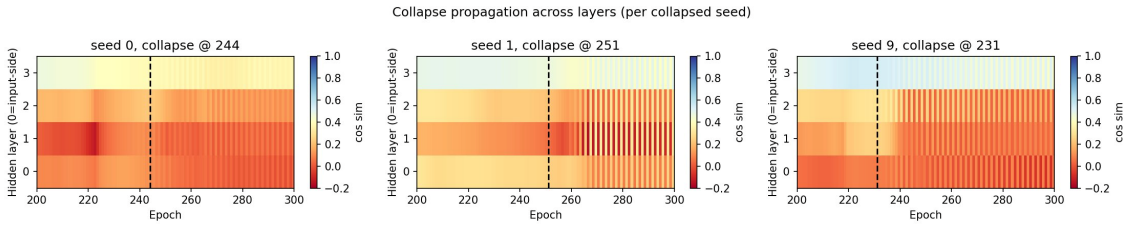

Figure 7: Per-seed collapse propagation across layers. Cosine similarity over epochs 200–300 for the three collapsing seeds. The black dashed line is the per-seed detected collapse epoch. The output layer (top row) carries the high-cosine signal that drops at collapse; the input-side layers (bottom rows) sit at low cosine throughout. The collapse propagates inward, but the input-side layers were never coherently aligned to begin with.

It is worth pausing on what this means. The standard story about why deep SNNs are hard to train is that the surrogate gradient is "biased", it points in roughly the wrong direction, and the bias compounds with depth. Our per-layer decomposition puts a sharper edge on this. The bias is not uniform across layers. The input-side layers receive a gradient signal that is essentially uncorrelated with the true gradient direction *from the very first epoch*. They are not deteriorating during training; they were never aligned to begin with. The reason this does not destroy training is that they also carry very little gradient mass: the output-side readout dominates the parameter update, and the output-side readout maintains a respectable cos ≈ 0.55 alignment for most of the trajectory.

This refines the picture in Deng et al. (2023), who frame the SG as inducing an intrinsic gradient error that, in their words, "would be magnified through layer wise backpropagation, especially through multiple layers." Our per-layer cosine measurements are consistent with their compounding picture, input-side layers do show much weaker alignment than the readout, and the per-layer mean true/surrogate derivative ratio decreases steadily from layer 0 to layer 3 (improvement #1 in Phase 4). But the gradient-mass-weighted average (Section 5, Equation above) shows the compounding does not translate into a correspondingly large global error: input-side layers carry too little gradient mass to drag the global cosine far from the output-side cosine. The Deng et al. mechanism is real, but its global effect is muted by the gradient-mass distribution, which is also why their proposed fix (shortcut surrogate modules to deliver more accurate gradient to upstream layers) operates exactly where the unfixed gradient signal is weakest.

The most important consequence is what it implies for the loss curve. Figure 2 (bottom) shows that training loss continues to decrease *through* the collapse, and at the three landscape checkpoints (Figure 5) the iterate sits in progressively softer quadratic basins with monotonically decreasing loss. The SG remains a useful descent direction even when its cosine with the true gradient has dropped near zero. This is a strengthening of the original Gygax and Zenke result, not a weakening: useful descent does not require directional alignment with the true gradient. The SG and the true gradient agree on what the output-side weights should do; they disagree about what the input-side weights should do; but the input-side weights' contribution to the loss is small enough that the disagreement does not block progress. The broader optimization literature already contains a precedent for this, Bernstein et al. (2018) show that signSGD, which discards gradient magnitude entirely and retains only the sign, achieves SGD-level convergence rates on non-convex problems. Our finding fits the same pattern from a different angle: SGD with a directionally-distorted-but-magnitude-preserving gradient (the SG) optimizes successfully, just as SGD with a sign-preserving-but-magnitude-discarded gradient (signSGD) does. The implication is that the ε-quality of a gradient required for descent is loose; both direction-preservation and magnitude-preservation are individually relaxable.

6 Stress-Tests of the Conclusions

The conclusions of Sections 4 and 5 each rest on choices that could be wrong. Phase 4 of the project consists of seven follow-up scripts, each designed to attack a specific weakness; we summarize the headline results below.

(i) Surrogate-steepness robustness. The static result is insensitive to β_{sg}, but the late-training collapse could still be a knob phenomenon. We sweep $\beta_{sg} \in \{2, 3, 5, 7, 10\}$ at 10 seeds each (Figure 8). The collapse rate is exactly flat at 3/10 across the entire sweep (95% bootstrap CI [0, 0.6]); the median collapse epoch drifts by only 3 epochs across the range; and the final cosine similarity drifts from 0.66 to 0.57 as β_{sg} increases from 2 to 10. Collapse is not a β_{sg}-knob phenomenon.

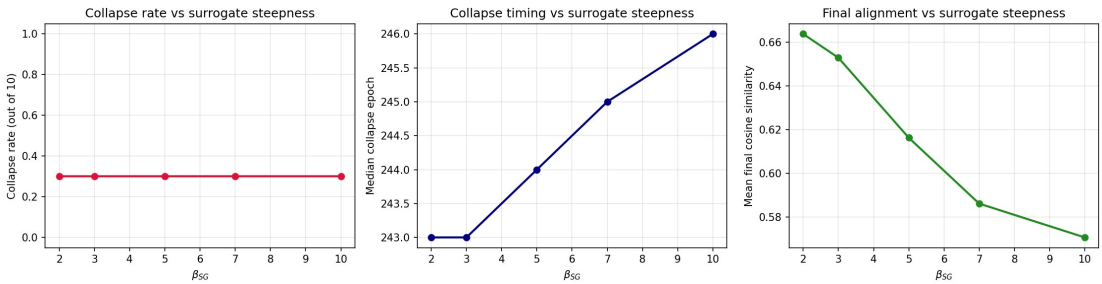

Figure 8: β_{sg} robustness sweep, 10 seeds per cell. Left: collapse rate is exactly flat at 3/10 across the full range. Centre: median collapse epoch drifts by only 3 epochs. Right: mean final cosine similarity drifts modestly (0.66 → 0.57) as β_{sg} increases.

This bears directly on prior approaches that prescribe adaptive adjustment of the surrogate steepness or relaxation degree as a remedy for the SG–true-gradient mismatch (Wang et al., 2023; Li et al., 2021). For the static metric reported by Gygax and Zenke (2024) and for the late-training *directional* collapse we identify here, varying β_{sg} over the range these methods consider does not change the collapse rate and barely shifts the collapse timing. Whatever lever closes the directional gap, β_{sg} is not it, at least for late-training collapse in deep sigmoid MLPs. The final-cosine drift visible in Figure 8 (right) is the only β_{sg} effect and is in the opposite direction one would predict from a smoothing-error story: shallower surrogates (smaller β_{sg}) give *higher* final alignment, not lower. This extends the static-task robustness finding of Zenke and Vogels (2021), that SG learning is robust to large changes in surrogate shape to the dynamic, late-training collapse phenomenon: the collapse, like task accuracy in their study, is essentially insensitive to surrogate steepness.

(ii) Power rerun for instrumentation. The original perturbation-sensitivity test (probed vs unprobed seeds) was just-marginal at $N = 30$ with McNemar $p = 0.0625$. Doubling to $N = 60$ (Figure 9) gives a paired difference of +0.117 with 95% bootstrap CI [+0.017, +0.217] (excludes zero) and McNemar $p = 0.0654$. Effect size halves but direction preserved: probing weakly *suppresses* collapse (probed-rate 0.42 < control-rate 0.53), and crucially never creates one, the off-diagonal cell (control: none, probed: collapse) holds 2 out of 60 paired seeds, far below the 9 in the symmetric off-diagonal cell.

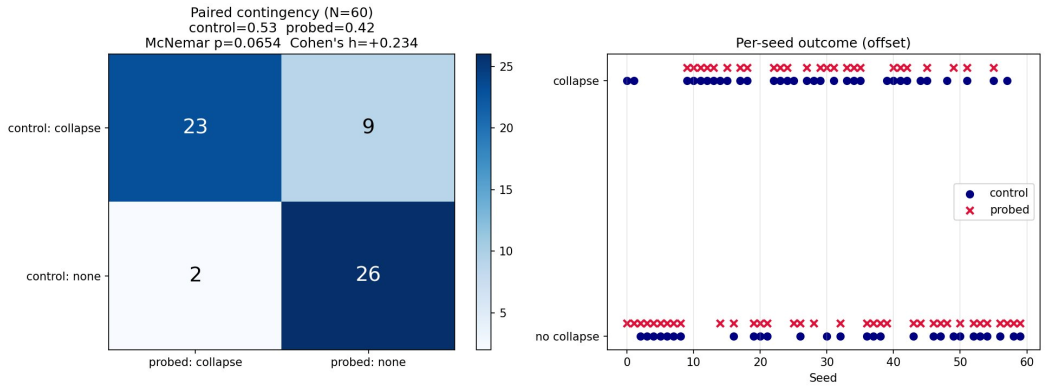

Figure 9: Paired probed-vs-unprobed test, $N = 60$. Left: 2×2 contingency table with paired control/probed rates 0.53 / 0.42 and McNemar $p = 0.0654$. Right: per-seed outcomes (offset for visibility); blue circles are the unprobed control arm, red crosses the probed arm. The asymmetry between cells (9, 2) of the contingency table is what drives the result.

(iii) Multi-scale and Hessian-eigenvector landscape. The original 1D landscape visualization used a single ±1.0 range, very large compared to the per-step parameter delta ($\sim 10^{-2}$). We rerun at four ranges {±0.05, ±0.1, ±0.5, ±1.0} and replace random directions with the top two filter-normalized Hessian eigenvectors. At every zoom and at every collapse-window epoch the iterate sits in a smooth quadratic basin (Figure 5 above), and λ_{max} decreases monotonically across the window, the cleanest single piece of evidence that the collapse is not a sharpening event.

(iv) Stricter saturation rule. The original saturation analysis used an output-magnitude rule; the SG-vs-true comparison cares about derivative magnitude, not output magnitude. We replace the rule with three derivative-based thresholds at $\varepsilon \in \{0.01, 0.05, 0.10\}$ of $\beta_f/4$ and additionally track the per-layer mean true/surrogate derivative ratio. The saturation fraction remains flat at 0.88 across the board, and Pearson r(saturated, cosine) $\in \{-0.030, 0.014, 0.034\}$. The Phase-3 rejection of saturation survives. **(v) Per-layer collapse detector.** Re-running the same causal collapse detector on each layer's trajectory, only 3 of 40 (10 seeds × 4 layers) trajectories register a collapse, and none align with the global collapse epoch. **(vi) Bootstrap CIs.** Replacing every Welch t-test with a $B = 10{,}000$ nonparametric bootstrap (paired resampling for paired tests) leaves every conclusion unchanged: GPU and CPU λ_{max} CIs are indistinguishable; both perturbation-difference CIs strictly exclude zero.

7 Limitations and Barriers

The clearest barrier we hit was statistical power. The original perturbation-sensitivity test (Section 6, item ii) was just-marginal at $N = 30$ with McNemar $p \approx 0.06$, and only the doubled-N rerun gave a bootstrap CI that excluded zero. The Welch t-test on λ_{max} pre vs post collapse ($t = -2.04$, $p \approx 0.094$) similarly only became convincing when bootstrap CIs were added. We learned to plan power ahead, and to default to bootstrap CIs over t-tests at small N.

A second barrier was the boundary artifact of `np.convolve(mode='same')` in the original collapse detector, which produced a fake universal collapse at the tail of every trajectory and very nearly led us to the wrong qualitative conclusion. Causal trailing-cumsum smoothing fixed it, but the lesson, that smoothing-kernel choice can manufacture the phenomenon you are trying to detect, applied to several later analyses as well.

The most important external-validity caveat is the narrow scope of the experimental setup. All sigmoid-MLP results live on `make_moons` and `make_circles`; we have no image, tabular, or sequence benchmark, and no real-world data. The optimizer is full-batch SGD with learning rate 0.01; we do not test Adam, momentum, mini-batch noise, or learning-rate schedules, all of which are known to interact non-trivially with curvature. The architecture is a sigmoid-activation MLP at width 64 and depths 2–6; we do not test ReLU, normalization, residual connections, or attention. A run at lr $\ll$ 0.01 would also disambiguate whether the collapse is an effective-time phenomenon (number of effective parameter updates) or an epoch-count phenomenon. Finally, we have no theoretical chain-rule argument linking the output-side gradient agreement to a closed-form condition on the post-activation distribution; that would convert the mechanism in Section 5 from empirical to predictive.

8 Conclusion

The directional gap between the surrogate gradient and the true gradient is real, depth-driven, and qualitatively different across layers. Input-side layers in deep sigmoid MLPs are perpetually misaligned with the true gradient (cos ≈ 0.10 from the first epoch), but they carry little gradient mass; the output-side readout carries both the dominant gradient mass and a respectable cos ≈ 0.55. The late-training collapse we characterize is a transition in *which* layers contribute the dominant gradient mass, not a phase change in the loss surface, the curvature spectrum, the activity regime, or the hardware. Crucially, training loss continues to decrease through the collapse: the surrogate gradient remains a useful descent direction even when nearly orthogonal to the true gradient. We read this as a strengthening of the case for surrogate gradient methods in deep spiking networks. Useful descent does not require directional alignment with the true gradient, it requires directional alignment *where the gradient mass is*, which in deep sigmoid MLPs is the output side.

Code availability. All scripts, experiment configurations, and per-seed outputs are available at: https://github.com/GladwinO/Algorithmic-ML-Final-Project.

References

Gygax, J. and Zenke, F. (2024). Elucidating the theoretical underpinnings of surrogate gradient learning in spiking neural networks. *arXiv preprint*.

Bellec, G., Salaj, D., Subramoney, A., Legenstein, R., and Maass, W. (2018). Long short-term memory and learning-to-learn in networks of spiking neurons. *Advances in Neural Information Processing Systems (NeurIPS)*, 31.

Zenke, F. and Vogels, T. P. (2021). The remarkable robustness of surrogate gradient learning for instilling complex function in spiking neural networks. *Neural Computation*, 33(4), 899–925.

Li, Y., Guo, Y., Zhang, S., Deng, S., Hai, Y., and Gu, S. (2021). Differentiable Spike: Rethinking gradient-descent for training spiking neural networks. *Advances in Neural Information Processing Systems (NeurIPS)*, 34.

Deng, S., Lin, H., Li, Y., and Gu, S. (2023). Surrogate Module Learning: Reduce the gradient error accumulation in training spiking neural networks. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, PMLR 202, 7645–7657.

Wang, Z., Jiang, R., Lian, S., Yan, R., and Tang, H. (2023). Adaptive Smoothing Gradient Learning for spiking neural networks. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, PMLR 202, 35798–35816.

Neftci, E. O., Mostafa, H., and Zenke, F. (2019). Surrogate gradient learning in spiking neural networks: Bringing the power of gradient-based optimization to spiking neural networks. *IEEE Signal Processing Magazine*, 36(6), 51–63.

Sankararaman, K. A., De, S., Xu, Z., Huang, W. R., and Goldstein, T. (2020). The impact of neural network overparameterization on gradient confusion and stochastic gradient descent. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, PMLR 119, 8469–8479.

Bernstein, J., Wang, Y.-X., Azizzadenesheli, K., and Anandkumar, A. (2018). signSGD: Compressed Optimisation for Non-Convex Problems. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, PMLR 80, 559–568.

Li, H., Xu, Z., Taylor, G., Studer, C., and Goldstein, T. (2018). Visualizing the loss landscape of neural nets. *Advances in Neural Information Processing Systems (NeurIPS)*, 31.

Hutchinson, M. F. (1990). A stochastic estimator of the trace of the influence matrix for Laplacian smoothing splines. *Communications in Statistics — Simulation and Computation*, 19(2), 433–450.

Lanczos, C. (1950). An iteration method for the solution of the eigenvalue problem of linear differential and integral operators. *Journal of Research of the National Bureau of Standards*, 45(4), 255–282.

Paszke, A. et al. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems (NeurIPS)*, 32, 8024–8035.

Pedregosa, F. et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

McNemar, Q. (1947). Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2), 153–157.

Efron, B. (1979). Bootstrap methods: another look at the jackknife. *The Annals of Statistics*, 7(1), 1–26.